TOPIC 3 : DIVIDE AND CONQUER

1. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N= 8, a[] = {5,7,3,4,9,12,6,2}

Output : Min = 2, Max = 12

Test Cases :

Input : N= 9, a[] = {1,3,5,7,9,11,13,15,17}

Output : Min = 1, Max = 17

Test Cases :

Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17}

Output : Min 12, Max 67

Aim

To write a Python program to find the **minimum** and **maximum** values in a given array.

Algorithm

1. Start.
2. Read the number of elements N and the array.
3. Assume the first element as both the minimum and maximum.
4. Traverse the remaining elements of the array.
5. If an element is smaller than the current minimum, update the minimum.
6. If an element is greater than the current maximum, update the maximum.
7. Display the minimum and maximum values.
8. Stop.

code:

```
# Function to find minimum and maximum values
def findMinMax(arr):
    minimum = arr[0]
    maximum = arr[0]
    for i in range(1, len(arr)):
        if arr[i] < minimum:
            minimum = arr[i]
        if arr[i] > maximum:
            maximum = arr[i]
    return minimum, maximum
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
arr = list(map(int, input().split()))
# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    minimum, maximum = findMinMax(arr)
    print("Min =", minimum)
    print("Max =", maximum)
```

input and output:

```
Enter the number of elements: 8
Enter the array elements:
5 7 3 4 9 12 6 2
Min = 2
Max = 12
```

- Time Complexity **: O(n)**
- Space Complexity **: O(1)**

Result

The program successfully finds and displays the **minimum** and **maximum** values in the given array.

2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N=8, 2,4,6,8,10,12,14,18.

Output : Min = 2, Max =18

Test Cases :

Input : N= 9, a[] = {11,13,15,17,19,21,23,35,37}

Output : Min = 11, Max = 37

Test Cases :

Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17}

Output : Min 12, Max 67

Aim

To write a Python program to find the **minimum** and **maximum** values in a given array.

Algorithm

1. Start.
2. Read the number of elements N and the array.
3. Assume the first element as both the minimum and maximum.
4. Traverse the remaining elements of the array.
5. If an element is smaller than the current minimum, update the minimum.
6. If an element is greater than the current maximum, update the maximum.
7. Display the minimum and maximum values.
8. Stop.

code:

```python
# Function to find minimum and maximum values
def findMinMax(arr):
    minimum = arr[0]
    maximum = arr[0]
    for i in range(1, len(arr)):
        if arr[i] < minimum:
            minimum = arr[i]
        if arr[i] > maximum:
            maximum = arr[i]
    return minimum, maximum
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
arr = list(map(int, input().split()))
# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    minimum, maximum = findMinMax(arr)
    print("Min =", minimum)
    print("Max =", maximum)
```

input and output:

```
Enter the number of elements: 8
Enter the array elements:
2 4 6 8 10 12 14 18
Min = 2
Max = 18
```

- Time Complexity : **O(n)**
- Space Complexity : **O(1)**

Result

The program successfully finds and displays the **minimum** and **maximum** values in the given array.

3. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice.

Test Cases :

Input : N= 8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35

Test Cases :

Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Aim

To write a Python program to sort an **unsorted array** in **ascending order** using the **Merge Sort** algorithm.

Algorithm

1. Start.
2. Read the number of elements N and the array.
3. If the array has only one element, it is already sorted.
4. Divide the array into two halves.
5. Recursively apply Merge Sort to both halves.
6. Merge the two sorted halves into a single sorted array.
7. Display the sorted array.
8. Stop.

code:

```python
# Function to perform Merge Sort
def mergeSort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        # Recursively sort both halves
        mergeSort(left)
        mergeSort(right)
        i = j = k = 0
        # Merge the sorted halves
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        # Copy remaining elements of left half
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        # Copy remaining elements of right half
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
```

input and output:

```
Enter the number of elements: 8
Enter the array elements:
31 23 35 27 11 21 15 28
Sorted Array: [11, 15, 21, 23, 27, 28, 31, 35]
```

- Time Complexity**: O(n log n)**
- Space Complexity**: O(n)**

Result

The program successfully sorts the given array in **ascending order** using the **Merge Sort** algorithm.

4. Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

Test Cases :

Input : N= 8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89

Test Cases :

Input : N= 7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82,

Aim

To write a Python program to sort an array using the **Merge Sort** algorithm and count the **number of comparisons** made during the sorting process.

Algorithm

1. Start.
2. Read the number of elements N and the array.
3. Divide the array into two halves until each subarray contains one element.
4. Merge the subarrays:
   - Compare the first elements of both subarrays.
   - Count each comparison.
   - Place the smaller element into the merged array.
5. Continue merging until the entire array is sorted.
6. Display the sorted array.
7. Display the total number of comparisons.
8. Stop.

code:

```
# Global variable to count comparisons
comparisons = 0
# Function to perform Merge Sort
def mergeSort(arr):
    global comparisons
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        # Recursively sort both halves
        mergeSort(left)
        mergeSort(right)
        i = j = k = 0
        # Merge the sorted halves
        while i < len(left) and j < len(right):
            comparisons += 1  # Count comparison
            if left[i] <= right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        # Copy remaining elements
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
```

```
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
arr = list(map(int, input().split()))
# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    mergeSort(arr)
    print("Sorted Array:", arr)
    print("Number of Comparisons:", comparisons)
```

input and output:

```
Enter the number of elements: 8
Enter the array elements:
12 4 78 23 45 67 89 1
Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89]
Number of Comparisons: 16
```

- Time Complexity **: O(n log n)**
- Space Complexity **: O(n)**

Result

The program successfully sorts the given array using the **Merge Sort** algorithm and displays the **sorted array** along with the **total number of comparisons** performed during the merge process.

5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

Input : N= 9, a[]= {10,16,8,12,15,6,3,9,5}

Output : 3,5,6,8,9,10,12,15,16

Test Cases :

Input : N= 8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89

Test Cases :

Input : N= 7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82,

Aim

To write a Python program to sort an **unsorted array** using the **Quick Sort** algorithm by choosing the **first element as the pivot** and recursively sorting the sub-arrays.

Algorithm

1. Start.
2. Read the number of elements N and the array.
3. Choose the **first element** as the pivot.
4. Partition the array so that:
   - Elements smaller than the pivot are placed to its left.
   - Elements greater than the pivot are placed to its right.
5. Place the pivot in its correct sorted position.
6. Recursively apply Quick Sort to the left and right sub-arrays.
7. Repeat until all sub-arrays contain one or no elements.
8. Display the sorted array.
9. Stop.

code:

```python
# Function to partition the array using the first element as pivot
def partition(arr, low, high):
    pivot = arr[low]
    i = low + 1
    j = high
    while True:
        while i <= high and arr[i] <= pivot:
            i += 1
        while arr[j] > pivot:
            j -= 1
        if i < j:
            arr[i], arr[j] = arr[j], arr[i]
        else:
            break
    arr[low], arr[j] = arr[j], arr[low]
    return j
# Quick Sort function
def quickSort(arr, low, high):
    if low < high:
        p = partition(arr, low, high)
        print("After partition:", arr)
        quickSort(arr, low, p - 1)
        quickSort(arr, p + 1, high)
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
arr = list(map(int, input().split()))
# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    print("Original Array:", arr)
```

input and output:

```
Enter the number of elements: 7
Enter the array elements:
38 27 43 3 9 82 10
Original Array: [38, 27, 43, 3, 9, 82, 10]
After partition: [9, 27, 10, 3, 38, 82, 43]
After partition: [3, 9, 10, 27, 38, 82, 43]
After partition: [3, 9, 10, 27, 38, 82, 43]
After partition: [3, 9, 10, 27, 38, 43, 82]
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
```

- Time Complexity **: Best/Average: O(n log n), Worst: O(n^2)**
- Space Complexity **: O(log n)**

Result

The program successfully sorts the given array in **ascending order** using the **Quick Sort** algorithm with the **first element as the pivot**.

6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array.

Input : N= 8, a[] = {19,72,35,46,58,91,22,31}

Output : 19,22,31,35,46,58,72,91

Test Cases :

Input : N= 8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35

Test Cases :

Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Aim

To write a Python program to sort an **unsorted array** using the **Quick Sort** algorithm by choosing the **middle element as the pivot** and recursively sorting the sub-arrays.

Algorithm

1. Start.
2. Read the number of elements N and the array.
3. Choose the **middle element** as the pivot.
4. Partition the array into three parts:
   - Elements smaller than the pivot.
   - Elements equal to the pivot.
   - Elements greater than the pivot.
5. Recursively apply Quick Sort to the left and right sub-arrays.
6. Combine the sorted left sub-array, pivot elements, and sorted right sub-array.
7. Display the sorted array.
8. Stop.

code:

```python
# Function to perform Quick Sort using middle element as pivot
def quickSort(arr):
    if len(arr) <= 1:
        return arr
    # Choose middle element as pivot
    pivot = arr[len(arr) // 2]
    left = []
    middle = []
    right = []
    # Partition the array
    for x in arr:
        if x < pivot:
            left.append(x)
        elif x == pivot:
            middle.append(x)
        else:
            right.append(x)
    print("After partition:", left + middle + right)
    # Recursively sort left and right sub-arrays
    return quickSort(left) + middle + quickSort(right)
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
arr = list(map(int, input().split()))
# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    print("Original Array:", arr)
    sortedArray = quickSort(arr)
    print("Sorted Array:", sortedArray)
```

input and output:

```
Enter the number of elements: 8
Enter the array elements:
19 72 35 46 58 91 22 31
Original Array: [19, 72, 35, 46, 58, 91, 22, 31]
After partition: [19, 35, 46, 22, 31, 58, 72, 91]
After partition: [19, 35, 22, 31, 46]
After partition: [19, 22, 35, 31]
After partition: [31, 35]
After partition: [72, 91]
Sorted Array: [19, 22, 31, 35, 46, 58, 72, 91]
```

- Time Complexity **: Best/Average: O(n log n), Worst: O(n²)**
- Space Complexity **: O(log n)**

Result

The program successfully sorts the given array in **ascending order** using the **Quick Sort** algorithm with the **middle element as the pivot**.

7. Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.

Input : N= 9, a[] = {5,10,15,20,25,30,35,40,45}, search key = 20

Output : 4

Test cases

Input : N= 6, a[] = {10,20,30,40,50,60}, search key = 50

Output : 5

Input : N= 7, a[] = {21,32,40,54,65,76,87}, search key = 32

Output : 2

Aim

To write a Python program to search for a given element in a **sorted array** using the **Binary Search** algorithm and count the **number of comparisons** made during the search.

Algorithm

1. Start.
2. Read the number of elements N, the sorted array, and the search key.
3. Initialize low = 0 and high = N - 1.
4. Initialize comparisons = 0.
5. Repeat while low <= high:
   - Find the middle index mid.
   - Increment the comparison count.
   - If a[mid] equals the search key, display its position and the comparison count.
   - If the search key is smaller than a[mid], search the left half.
   - Otherwise, search the right half.
6. If the element is not found, display **"Element not found"**.
7. Stop.

code:

```
# Function to perform Binary Search
def binarySearch(arr, key):
    low = 0
    high = len(arr) - 1
    comparisons = 0
    while low <= high:
        comparisons += 1
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid, comparisons
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comparisons
# User input
n = int(input("Enter the number of elements: "))
print("Enter the sorted array elements:")
arr = list(map(int, input().split()))
key = int(input("Enter the search key: "))
# Check if the correct number of elements is entered
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    index, comparisons = binarySearch(arr, key)
    if index != -1:
        print("Element found at position:", index + 1)
    else:
        print("Element not found.")
    print("Number of Comparisons:", comparisons)
```

input and output:

```
Enter the number of elements: 6
Enter the sorted array elements:
10 20 30 40 50 60
Enter the search key: 50
Element found at position: 5
Number of Comparisons: 2
```

- Time Complexity : **O(log n)**
- Space Complexity : **O(1)**

Result

The program successfully searches for the given element using the **Binary Search** algorithm and displays its **position** along with the **number of comparisons** performed during the search.

8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?

Input : N= 9, a[] = {3,9,14,19,25,31,42,47,53}, search key = 31

Output : 6

Test cases

Input : N= 7, a[] = {13,19,24,29,35,41,42}, search key = 42

Output : 7

Test cases

Input : N= 6, a[] = {20,40,60,80,100,120}, search key = 60

Output : 3

Aim

To write a Python program to search for an element in a **sorted array** using the **Binary Search** algorithm and demonstrate the **mid-point calculations** involved in the search.

Algorithm

1. Start.
2. Read the number of elements N, the sorted array, and the search key.
3. Initialize low = 0 and high = N - 1.
4. Repeat while low <= high:
   - Calculate mid = (low + high) / 2.
   - If a[mid] equals the search key, display its position and stop.
   - If the search key is smaller than a[mid], set high = mid - 1.
   - Otherwise, set low = mid + 1.
5. If the element is not found, display **"Element not found"**.
6. Stop.

code:

```
# Function to perform Binary Search with steps
def binarySearch(arr, key):
    low = 0
    high = len(arr) - 1
    comparisons = 0
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
        if arr[mid] == key:
            return mid, comparisons
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comparisons
# User input
n = int(input("Enter the number of elements: "))
print("Enter the sorted array elements:")
arr = list(map(int, input().split()))
key = int(input("Enter the search key: "))
# Check input size
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    index, comparisons = binarySearch(arr, key)
    if index != -1:
        print("Element found at position:", index + 1)
    else:
        print("Element not found.")
    print("Number of Comparisons:", comparisons)
```

input and output:

```
Enter the number of elements: 6
Enter the sorted array elements:
20 40 60 80 100 120
Enter the search key: 60
Element found at position: 3
Number of Comparisons: 1
```

- Time Complexity : **O(log n)**
- Space Complexity : **O(1)**

Effect of an Unsorted Array

Binary Search requires the array to be sorted.

If the array is not sorted:

- The middle element cannot correctly determine which half contains the search key.
- The algorithm may search the wrong half.
- The element may not be found even if it exists.
- Therefore, the result is incorrect.

Performance Impact:

- Binary Search still performs O(log n) comparisons, but the result is unreliable because the search decisions are invalid.
- To use Binary Search correctly on an unsorted array, the array must first be sorted, which takes O(n log n) time.

Result

The program successfully searches for the required element in a sorted array using the Binary Search algorithm and demonstrates the midpoint calculations at each step.

9. Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

Input : points = [[1,3],[-2,2],[5,8],[0,1]],k=2

Output:[[-2, 2], [0, 1]]

(ii) Input: points = [[1, 3], [-2, 2]], k = 1

Output: [[-2, 2]]

(iii) Input: points = [[3, 3], [5, -1], [-2, 4]], k = 2

Output: [[3, 3], [-2, 4]]

Aim

To write a Python program to find the **k closest points to the origin (0, 0)** from a given set of points on the X-Y plane.

Algorithm

1. Start.
2. Read the list of points and the value of k.
3. For each point (x, y):
   - Calculate its distance from the origin using the Euclidean distance formula:
     - sqrt(X^2+y^2)
4. Sort the points based on their distances from the origin.
5. Select the first k points from the sorted list.
6. Display the selected points.
7. Stop.

code:

```python
import math
# Function to find k closest points
def kClosest(points, k):
    distances = []
    # Calculate distance of each point from origin
    for point in points:
        x, y = point
        distance = math.sqrt(x * x + y * y)
        distances.append((distance, point))
    # Sort based on distance
    distances.sort()
    # Store first k points
    result = []
    for i in range(k):
        result.append(distances[i][1])
    return result
# User input
n = int(input("Enter the number of points: "))
points = []
print("Enter the coordinates (x y):")
for i in range(n):
    x, y = map(int, input().split())
    points.append([x, y])
k = int(input("Enter the value of k: "))
result = kClosest(points, k)
print("Output:", result)
```

input and output:

```
Enter the number of points: 2
Enter the coordinates (x y):
1 3
-2 2
Enter the value of k: 1
Output: [[-2, 2]]
```

- Time Complexity : **O(n log n)**
- Space Complexity : **O(n)**

Result

The program successfully finds and displays the **k closest points to the origin** by calculating the distance of each point and selecting the points with the smallest distances.

10. Given four lists A, B, C, D of integer values, Write a program to compute how many tuples n(i, j, k, l) there are such that A[i] + B[j] + C[k] + D[l] is zero.

Input: A = [1, 2], B = [-2, -1], C = [-1, 2], D = [0, 2]

Output: 2

(ii) Input: A = [0], B = [0], C = [0], D = [0]

Output: 1

Aim

To write a Python program to find the **number of tuples** (i, j, k, l) such that:

A[i]+B[j]+C[k]+D[l]=0

Algorithm

1. Start.
2. Read the four integer arrays A, B, C, and D.
3. Create a hash map to store the sums of every pair of elements from arrays A and B along with their frequencies.
4. Traverse every pair of elements from arrays C and D.
5. Compute the negative of their sum.
6. If this value exists in the hash map, add its frequency to the count.
7. After checking all pairs, display the total count.
8. Stop.

code:

```
# Function to count tuples whose sum is zero
def fourSumCount(A, B, C, D):
    count = 0
    # Check all possible tuples
    for a in A:
        for b in B:
            for c in C:
                for d in D:
                    if a + b + c + d == 0:
                        count += 1
    return count
# User input
n = int(input("Enter the number of elements in each array: "))
print("Enter elements of array A:")
A = list(map(int, input().split()))
print("Enter elements of array B:")
B = list(map(int, input().split()))
print("Enter elements of array C:")
C = list(map(int, input().split()))
print("Enter elements of array D:")
D = list(map(int, input().split()))
# Check input size
if len(A) != n or len(B) != n or len(C) != n or len(D) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    result = fourSumCount(A, B, C, D)
    print("Output:", result)
```

input and output:

```
Enter the number of elements in each array: 2
Enter elements of array A:
1 2
Enter elements of array B:
-2 -1
Enter elements of array C:
-1 2
Enter elements of array D:
0 2
Output: 2
```

- Time Complexity : **$O(n^2)$**
- Space Complexity : **$O(n^2)$**

Result

The program successfully computes the **number of tuples** (i, j, k, l) such that the sum of the corresponding elements from arrays A, B, C, and D is equal to **zero**.

11. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array.

arr = [12, 3, 5, 7, 19] k = 2 Expected Output:5

arr = [12, 3, 5, 7, 4, 19, 26] k = 3 Expected Output:5

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6 Expected Output:6

Aim

To write a Python program to find the **k-th smallest element** in an unsorted array using the **Median of Medians** algorithm, which guarantees **worst-case linear time complexity**.

Algorithm

1. Start.
2. Read the array and the value of k.
3. Divide the array into groups of 5 elements.
4. Sort each group and find its median.
5. Collect all medians into a new list.
6. Recursively find the median of the medians and use it as the pivot.
7. Partition the array into:
   - Elements smaller than the pivot.
   - Elements equal to the pivot.
   - Elements greater than the pivot.
8. Depending on the value of k:
   - Return the pivot if it is the k-th smallest.
   - Otherwise, recursively search the appropriate partition.
9. Display the k-th smallest element.
10. Stop.

code:

```python
# Function to find the median of a small list
def findMedian(arr):
    arr.sort()
    return arr[len(arr) // 2]
# Median of Medians Algorithm
def kthSmallest(arr, k):
    if len(arr) <= 5:
        arr.sort()
        return arr[k - 1]
    # Divide the array into groups of 5
    groups = [arr[i:i + 5] for i in range(0, len(arr), 5)]
    # Find the median of each group
    medians = [findMedian(group) for group in groups]
    # Find the median of medians
    pivot = kthSmallest(medians, (len(medians) + 1) // 2)
    # Partition the array
    low = []
    high = []
    equal = []
    for num in arr:
        if num < pivot:
            low.append(num)
        elif num > pivot:
            high.append(num)
        else:
            equal.append(num)
    # Recursively find the kth smallest element
    if k <= len(low):
        return kthSmallest(low, k)
    elif k <= len(low) + len(equal):
        return pivot
    else:
        return kthSmallest(high, k - len(low) - len(equal))
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
arr = list(map(int, input().split()))
k = int(input("Enter the value of k: "))
# Check input size
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
elif k < 1 or k > n:
    print("Invalid value of k.")
else:
    result = kthSmallest(arr, k)
    print("Output:", result)
```

input and output:

```
Enter the number of elements: 5
Enter the array elements:
12 3 5 7 19
Enter the value of k: 2
Output: 5
```

- Time Complexity : **O(n)**
- Space Complexity : **O(n)**

Result

The program successfully finds the **k-th smallest element** in the given unsorted array using the **Median of Medians** algorithm, ensuring an efficient **worst-case linear-time** solution.

12. To Implement a function median of medians (arr, k) that takes an unsorted array arr and an integer k, and returns the k-th smallest element in the array.

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6

arr = [23, 17, 31, 44, 55, 21, 20, 18, 19, 27] k = 5

Output: An integer representing the k-th smallest element in the array.

Aim

To write a Python program to implement the **Median of Medians** algorithm to find the **k-th smallest element** in an unsorted array.

Algorithm

1. Start.
2. Read the array arr and the integer k.
3. If the array contains 5 or fewer elements:
   - Sort the array.
   - Return the (k−1)th element.
4. Divide the array into groups of 5 elements.
5. Find the median of each group.
6. Recursively find the median of the medians and use it as the pivot.
7. Partition the array into:
   - Elements smaller than the pivot.
   - Elements equal to the pivot.
   - Elements greater than the pivot.
8. Recursively search the appropriate partition based on the value of k.
9. Display the k-th smallest element.
10. Stop.

code:

```python
# Function to find the median of a small list
def findMedian(arr):
    arr.sort()
    return arr[len(arr) // 2]
# Median of Medians function
def median_of_medians(arr, k):
    if len(arr) <= 5:
        arr.sort()
        return arr[k - 1]
    # Divide array into groups of 5
    groups = [arr[i:i + 5] for i in range(0, len(arr), 5)]
    # Find medians of each group
    medians = [findMedian(group) for group in groups]
    # Find median of medians
    pivot = median_of_medians(medians, (len(medians) + 1) // 2)
    # Partition the array
    low = []
    equal = []
    high = []
    for x in arr:
        if x < pivot:
            low.append(x)
        elif x > pivot:
            high.append(x)
        else:
            equal.append(x)
```

```python
    # Find the k-th smallest element
    if k <= len(low):
        return median_of_medians(low, k)
    elif k <= len(low) + len(equal):
        return pivot
    else:
        return median_of_medians(high, k - len(low) - len(equal))
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
arr = list(map(int, input().split()))
k = int(input("Enter the value of k: "))
# Check input
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
elif k < 1 or k > n:
    print("Invalid value of k.")
else:
    result = median_of_medians(arr, k)
    print("Output:", result)
```

input and output:

```
Enter the number of elements: 10
Enter the array elements:
1 2 3 4 5 6 7 8 9 10
Enter the value of k: 6
Output: 6
```

- Time Complexity : **O(n)**
- Space Complexity : **O(n)**

Result

The program successfully finds the **k-th smallest element** in the given unsorted array using the **Median of Medians** algorithm.

13. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet in the Middle technique to efficiently find this subset.

a) Set[] = {45, 34, 4, 12, 5, 2} Target Sum : 42

b) Set[]= {1, 3, 2, 7, 4, 6} Target sum = 10:

Aim

To write a Python program to find the **subset whose sum is closest to a given target sum** using the **Meet in the Middle** technique.

Algorithm

1. Start.
2. Read the array of integers and the target sum.
3. Divide the array into two equal halves.
4. Generate all possible subset sums for the left half.
5. Generate all possible subset sums for the right half.
6. Sort the subset sums of the right half.
7. For each subset sum in the left half:
    - Find the subset sum in the right half that makes the total closest to the target.
8. Keep track of the closest sum and the corresponding subset.
9. Display the subset and its sum.
10. Stop.

code:

```python
from itertools import combinations
import bisect
# Function to generate all subset sums
def subsetSums(arr):
    sums = []
    for r in range(len(arr) + 1):
        for subset in combinations(arr, r):
            sums.append((sum(subset), list(subset)))
    return sums
# Meet in the Middle function
def meetInMiddle(arr, target):
    n = len(arr)
    # Divide the array into two halves
    right = arr[n // 2:]
    leftSums = subsetSums(left)
    rightSums = subsetSums(right)
    # Sort right subset sums
    rightSums.sort(key=lambda x: x[0])
    rightValues = [x[0] for x in rightSums]
    bestSum = float('inf')
    bestSubset = []
    for leftSum, leftSubset in leftSums:
        remaining = target - leftSum
        pos = bisect.bisect_left(rightValues, remaining)
```

(variable) rightSums: list

```python
        # Check nearest subset sums
        for idx in [pos - 1, pos]:
            if 0 <= idx < len(rightSums):
                total = leftSum + rightSums[idx][0]
                if abs(target - total) < abs(target - bestSum):
                    bestSum = total
                    bestSubset = leftSubset + rightSums[idx][1]
    return bestSubset, bestSum
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
arr = list(map(int, input().split()))
target = int(input("Enter the target sum: "))
# Check input
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    subset, total = meetInMiddle(arr, target)
    print("Closest Subset:", subset)
    print("Closest Sum:", total)
```

input and output:

```
Enter the number of elements: 6
Enter the array elements:
1 3 2 7 4 6
Enter the target sum: 10
Closest Subset: [4, 6]
Closest Sum: 10
```

- Time Complexity : **O(2^(n/2) × log(2^(n/2)))**
- Space Complexity : **O(2^(n/2))**

Result :

The program successfully finds the **subset whose sum is closest to the given target** using the **Meet in the Middle** technique.

14. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false.

a) E = {1, 3, 9, 2, 7, 12} exact Sum = 15

b) E = {3, 34, 4, 12, 5, 2} exact Sum = 15

Aim

To write a Python program to determine whether there exists a subset whose sum is exactly equal to the given target sum using the Meet in the Middle technique**.**

Algorithm

1. Start.
2. Read the array **E** and the target sum.
3. Divide the array into two halves.
4. Generate all possible subset sums of the left half.
5. Generate all possible subset sums of the right half.
6. Sort the subset sums of the right half.
7. For each subset sum in the left half:
   - Check whether **target – leftSum** exists in the right-half subset sums.
8. If such a pair is found, print True.
9. Otherwise, print False.
10. Stop.

code:

```python
from itertools import combinations
import bisect
# Function to generate all subset sums
def generateSubsetSums(arr):
    sums = []
    for r in range(len(arr) + 1):
        for subset in combinations(arr, r):
            sums.append(sum(subset))
    return sums
# Meet in the Middle function
def meetInMiddle(arr, target):
    n = len(arr)
    # Divide array into two halves
    left = arr[:n // 2]
    right = arr[n // 2:]
    # Generate subset sums
    leftSums = generateSubsetSums(left)
    rightSums = generateSubsetSums(right)
    # Sort right subset sums
    rightSums.sort()
```

```python
    # Search for the required sum
    for s in leftSums:
        required = target - s
        index = bisect.bisect_left(rightSums, required)
        if index < len(rightSums) and rightSums[index] == required:
            return True
    return False
# User input
n = int(input("Enter the number of elements: "))
print("Enter the array elements:")
arr = list(map(int, input().split()))
target = int(input("Enter the exact sum: "))
# Check input
if len(arr) != n:
    print("Error: Number of elements does not match the specified size.")
else:
    result = meetInMiddle(arr, target)
    print("Output:", result)
```

input and output:

```
Enter the number of elements: 6
Enter the array elements:
1 3 9 2 7 12
Enter the exact sum: 15
Output: True
```

- Time Complexity : **O(2^(n/2) × log(2^(n/2)))**
- Space Complexity : **O(2^(n/2))**

Result :

The program successfully determines whether there exists a subset whose sum is **exactly equal** to the given target using the **Meet in the Middle** technique.

15 Given two 2×2 Matrices A and B

A=(1 7 B=(1 3

3 5) 7 5)

Use Strassen's matrix multiplication algorithm to compute the product matrix C

such that C=A×B.

Test Cases:

Consider the following matrices for testing your implementation:

Test Case 1:

A=(1 7 B=(6 8

3 5), 4 2)

Expected Output:

C=(18 14

62 66)

Aim

To write a Python program to multiply two **2 × 2 matrices** using **Strassen's Matrix Multiplication Algorithm**.

Algorithm

1. Start.
2. Read the two 2 × 2 matrices A and B.
3. Divide each matrix into four elements:
   - A = [[a, b], [c, d]]
   - B = [[e, f], [g, h]]
4. Compute the seven Strassen products:
   - M1 = (a + d)(e + h)
   - M2 = (c + d)e
   - M3 = a(f − h)
   - M4 = d(g − e)
   - M5 = (a + b)h
   - M6 = (c − a)(e + f)
   - M7 = (b − d)(g + h)

5. Compute the elements of the result matrix:
   - C11 = M1 + M4 − M5 + M7
   - C12 = M3 + M5
   - C21 = M2 + M4
   - C22 = M1 − M2 + M3 + M6
6. Display the product matrix C.
7. Stop.

code:

```python
# Function to perform Strassen's Matrix Multiplication
def strassen(A, B):
    # Matrix A elements
    a = A[0][0]
    b = A[0][1]
    c = A[1][0]
    d = A[1][1]
    # Matrix B elements
    e = B[0][0]
    f = B[0][1]
    g = B[1][0]
    h = B[1][1]
    # Strassen's 7 products
    P1 = a * (
    P2 = (a +
    P3 = (c + d) * e
    P4 = d * (g - e)
    P5 = (a + d) * (e + h)
    P6 = (b - d) * (g + h)
    P7 = (a - c) * (e + f)
```

(variable) d: Any

```python
    # Result matrix
    C11 = P5 + P4 - P2 + P6
    C12 = P1 + P2
    C21 = P3 + P4
    C22 = P1 + P5 - P3 - P7
    return [[C11, C12], [C21, C22]]
# User input
print("Enter the elements of Matrix A (2x2):")
A = []
for i in range(2):
    row = list(map(int, input().split()))
    A.append(row)
print("Enter the elements of Matrix B (2x2):")
B = []
for i in range(2):
    row = list(map(int, input().split()))
    B.append(row)
# Multiply matrices
C = strassen(A, B)
print("Product Matrix:")
for row in C:
    print(row)
```

input and output:

```
Enter the elements of Matrix A (2x2):
1 7
3 5
Enter the elements of Matrix B (2x2):
1 3
7 5
Product Matrix:
[50, 38]
[38, 34]
```

- Time Complexity : **O(n^2.807)**
- Space Complexity : **O(n²)**

Result :

The program successfully multiplies the given 2 × 2 matrices using Strassen's Matrix Multiplication Algorithm and displays the product matrix.

16 Given two integers X=1234 and Y=5678: Use the Karatsuba algorithm to compute the product Z=X x Y

Test Case 1:

Input: x=1234,y=5678

Expected Output: z=1234×5678=7016652

Aim

To write a Python program to multiply two large integers using the **Karatsuba Multiplication Algorithm**, which reduces the number of recursive multiplications and is faster than the conventional multiplication method.

Algorithm

1. Start.
2. Read the two integers X and Y.
3. If either number has only one digit, return their product.
4. Find the maximum number of digits and divide both numbers into two halves.
5. Split:
   - X = a × 10^m + b
   - Y = c × 10^m + d
6. Compute:
   - P1 = Karatsuba(a, c)
   - P2 = Karatsuba(b, d)
   - P3 = Karatsuba(a + b, c + d)
7. Calculate:
   - Middle = P3 − P1 − P2
8. Compute the final product:
   - Result = P1 × 10^(2m) + Middle × 10^m + P2
9. Display the product.
10. Stop.

code:

```python
# Function to perform Karatsuba Multiplication
def karatsuba(x, y):
    # Base case
    if x < 10 or y < 10:
        return x * y
    # Find the maximum length
    n = max(len(str(x)), len(str(y)))
    m = n // 2
    # Split the numbers
    high1 = x // (10 ** m)
    low1 = x % (10 ** m)
    high2 = y // (10 ** m)
    low2 = y % (10 ** m)
    # Recursive calls
    z0 = karatsuba(low1, low2)
    z1 = karatsuba(low1 + high1, low2 + high2)
    z2 = karatsuba(high1, high2)
    # Karatsuba formula
    return (z2 * (10 ** (2 * m))) + ((z1 - z2 - z0) * (10 ** m)) + z0
# User input
x = int(input("Enter the first number: "))
y = int(input("Enter the second number: "))
# Compute product
result = karatsuba(x, y)
print("Product:", result)
```

input and output:

```
Enter the first number: 1234
Enter the second number: 5678
Product: 7006652
```

- Time Complexity : **O(n^1.585)**
- Space Complexity : **O(n)**

Result :

The program successfully computes the product of two integers using the **Karatsuba Multiplication Algorithm**.